C++ OOP — Class Members & Memory Management

Today's Agenda

- Mutator / Inspector / Facilitator
- Destructor
- Constructor Member Initializer List
- `const` keyword
- References
- Copy Constructor
- Dynamic Memory Allocation
- Shallow Copy vs. Deep Copy
- Static Data Members & Static Member Functions

1. Constructors and Their Types

A **constructor** is a special member function that is automatically called when an object is created. It has the same name as the class and no return type.

Type	Description
Parameterless (user-defined)	Defined by programmer, takes no arguments
Parameterized	Takes arguments to initialize data members
Copy Constructor	Initializes a new object from an existing object
Default (compiler-generated)	Provided by compiler when no constructor is written

```
class Student {
    int rollNo;
    string name;
public:
    Student() { }                          // 1. User-defined parameterless
    Student(int r, string n) { ... }       // 2. Parameterized
    Student(const Student& s) { ... }      // 3. Copy constructor
    // If none written, compiler provides: Student() {}   // 4. Default
};
```

> **Key point:** The moment you define any constructor, the compiler **stops** generating the default constructor automatically.

2. Mutator (Setter)

A **mutator** is a member function used to **modify** a specific data member of an object without replacing the whole object.

Why use it?

- Enforces data hiding (data members are `private`)
- Allows validation logic before setting a value

Syntax:

```
void set<MemberName>(DataType value) {
    memberName = value;
}
```

Example:

```
class Contact {
    long mobileNo;
public:
    void setMobileNo(long mobileNo) {
        this->mobileNo = mobileNo;   // 'this->' distinguishes member from
parameter
    }
};
```

> **Note:** `this->mobileNo` refers to the object's data member. `mobileNo` (without `this->`) refers to the parameter.

---

3. Inspector (Getter)

An **inspector** is a member function used to **retrieve** the value of a specific data member.

Return type must match the data type of the member being returned.

Syntax:

```
DataType get<MemberName>() {
    return memberName;
}
```

Example:

```
class Contact {
    long mobileNo;
public:
```

```
    long getMobileNo() {
        return mobileNo;
    }
};
```

> **Best practice:** Mark inspectors as `const` so they cannot accidentally modify the object.
>
> ```
> long getMobileNo() const { return mobileNo; }
> ```

---

4. Facilitator

A **facilitator** is a member function that performs a **behavior or operation** on the object — it is neither a pure setter nor a pure getter.

Examples:

- `display()` — prints object data
- `calculateSalary()` — performs a computation using the object's state
- `applyDiscount(float percent)` — modifies the object based on logic

```
class Employee {
    string name;
    float salary;
public:
    void display() {                          // Facilitator
        cout << name << " earns " << salary;
    }
    void applyBonus(float percent) {          // Facilitator
        salary += salary * (percent / 100);
    }
};
```

---

5. Destructor

A **destructor** is a special member function that is automatically called when an object goes **out of scope** or is explicitly destroyed. Its primary purpose is to **release resources** (e.g., dynamically allocated memory).

Rules:

1. Same name as the class, prefixed with `~`
2. No return type (not even `void`)
3. No parameters
4. Cannot be overloaded — a class has exactly **one** destructor
5. Called automatically at the closing `}` of the scope where the object was created

Syntax:

```
~ClassName() {
    // cleanup code
}
```

Example:

```
class Buffer {
    int* data;
public:
    Buffer() {
        data = new int[100];          // Allocate memory
        cout << "Constructor called\n";
    }
    ~Buffer() {
        delete[] data;                // Free memory
        cout << "Destructor called\n";
    }
};

int main() {
    Buffer b;                         // Constructor called here
}                                     // Destructor called automatically here
```

> **Important:** If you allocate memory using `new` inside a constructor, always free it using `delete` inside the destructor to prevent **memory leaks**.

---

6. Constructor Member Initializer List

Instead of assigning values inside the constructor body, you can initialize data members **directly before the body executes** using an initializer list.

Syntax:

```
ClassName(parameters) : member1(value1), member2(value2) {
    // constructor body
}
```

Example:

```
class Rectangle {
    int width, height;
public:
    Rectangle(int w, int h) : width(w), height(h) {
```

```
        // body can be empty if all members are initialized above
    }
};
```

When is it mandatory?

- Initializing `const` data members
- Initializing reference (`&`) data members
- Calling a base class constructor (in inheritance)

```
class Circle {
    const float PI;      // Must use initializer list
    float& radius;       // Must use initializer list
public:
    Circle(float& r) : PI(3.14159f), radius(r) { }
};
```

> **Remember:** `const` and reference members **cannot** be assigned inside the constructor body — they must be initialized in the initializer list.

---

7. `const` in C++

The `const` keyword tells the compiler that the value **cannot be modified** after initialization.

7.1 Constant Variable

```
const int MAX = 100;
MAX = 200;   //  Compiler error
```

7.2 Constant Function Parameter

Prevents the function from modifying the argument:

```
void display(const string& name) {
    // name = "test";   Not allowed
    cout << name;
}
```

7.3 Constant Member Function

A `const` member function guarantees it will **not modify** any data member of the object:

```
class Box {
    int side;
public:
    int getSide() const {       // 'const' after parameter list
        return side;            // OK — reading only
        // side = 10;           //  Not allowed
    }
};
```

7.4 Constant Object

A `const` object can only call `const` member functions:

```
const Box b;
b.getSide();    //  OK — getSide() is const
b.setSide(5);   //  Error — setSide() is not const
```

---

8. Reference

A **reference** is an **alias** (another name) for an existing variable. It does not create a new copy — it refers to the same memory location.

Syntax:

```
DataType& refName = existingVariable;
```

Example:

```
int x = 10;
int& ref = x;    // ref is an alias for x

ref = 20;        // Modifies x directly
cout << x;       // Output: 20
```

Key rules:

- A reference **must be initialized** when declared
- Once bound to a variable, it **cannot be rebound** to another
- No separate memory is allocated for the reference

Common use — pass by reference:

```
void increment(int& n) {
    n++;          // Modifies the original variable
}
```

> **Reference vs Pointer:**
>
> | Reference | Pointer |
> |---|---|
> | Must be initialized | Can be uninitialized (but risky) |
> | Cannot be null | Can be `nullptr` |
> | Cannot be rebound | Can point to different addresses |
> | Cleaner syntax | More flexible but complex |

---

10. Dynamic Memory Allocation

Dynamic memory is allocated at **runtime** from the **heap**, unlike local variables which are on the **stack**.

Operators:

- `new` — allocates memory and returns a pointer
- `delete` — frees memory allocated by `new`

Single Object

```
int* p = new int(42);     // Allocate and initialize
cout << *p;               // Dereference to access value: 42
delete p;                 // Free memory
p = nullptr;              // Good practice after delete
```

Array

```
int* arr = new int[5];    // Allocate array of 5 ints
arr[0] = 10;
delete[] arr;             // Use delete[] for arrays
arr = nullptr;
```

Object on Heap

```
Student* s = new Student("Bob", 102);
s->display();             // Use -> to access members via pointer
delete s;
```

Stack vs. Heap:

Stack	Heap
Automatic lifetime	Manual lifetime (`new`/`delete`)
Limited size	Large (limited by RAM)
Fast allocation	Slightly slower
No memory leak risk	Memory leak if `delete` is missed

---

13. Pointer vs. Reference — Detailed Comparison

Both pointers and references are used to **indirectly access** a variable, but they behave very differently.

What is a Pointer?

A **pointer** is a variable that **stores the memory address** of another variable. It has its own memory location and can be changed to point to different variables.

```
int x = 10;
int* p = &x;      // p stores the address of x

cout << p;        // Prints address (e.g., 0x61ff08)
cout << *p;       // Dereference: prints value 10

*p = 99;          // Modifies x through pointer
cout << x;        // Output: 99
```

What is a Reference?

A **reference** is an **alias** — another name for the same variable. It does not store an address separately; it *is* the variable under a different name.

```
int x = 10;
int& ref = x;     // ref IS x — same memory location

ref = 99;         // Modifies x directly
cout << x;        // Output: 99
```

Side-by-Side Comparison

Feature	Pointer (*)	Reference (&)
Declaration	`int* p = &x;`	`int& r = x;`
Initialization	Optional (but risky if skipped)	**Mandatory** at declaration

Feature	Pointer (*)	Reference (&)
Can be `null`	Yes — `int* p = nullptr;`	No — must always refer to a valid variable
Can be rebound	Yes — can point to another variable	No — fixed to the same variable forever
Own memory	Yes — pointer itself occupies memory (4 or 8 bytes)	No — no separate memory allocated
Dereferencing needed	Yes — use `*p` to get value	No — use `ref` directly
Arithmetic supported	Yes — `p++` moves to next memory location	No — no pointer arithmetic
Used with `new`/`delete`	Yes — primary tool for dynamic memory	Rarely
Syntax complexity	More complex	Cleaner, safer
Null check required	Yes — always check before use	No — never null

Code Comparison

```
int x = 10, y = 20;

// --- Pointer ---
int* p = &x;
cout << *p;     // 10
p = &y;         //  Can rebind to y
cout << *p;     // 20
p = nullptr;    //  Can be null

// --- Reference ---
int& r = x;
cout << r;      // 10
// r = y;       // This does NOT rebind r to y — it copies value of y into x!
                // r is still referring to x, but x is now 20
cout << r;      // 20 (but r still refers to x)
```

> **Common student mistake:** Writing `r = y` after `int& r = x` does **not** make `r` refer to `y`. It copies the value of `y` into `x`. References cannot be rebound after initialization.

When to Use Which?

Situation	Use
Dynamic memory (`new`)	Pointer
Optional / nullable parameter	Pointer

Situation	Use
Arrays and pointer arithmetic	Pointer
Function parameter — avoid copy, never null	Reference
Return value aliasing	Reference
Operator overloading	Reference

---

14. `new` vs. `malloc` — Detailed Comparison

Both `new` and `malloc` allocate memory from the **heap** at runtime, but they differ significantly in behavior, especially in C++.

`malloc` (C-style)

`malloc` stands for **memory allocate**. It is a function from the C standard library (`<cstdlib>`). It allocates raw bytes and returns a `void*` pointer.

```
#include <cstdlib>

int* p = (int*)malloc(sizeof(int));   // Allocate 4 bytes
*p = 42;
free(p);                              // Release memory with free()
p = nullptr;
```

`new` (C++-style)

`new` is a C++ **operator** (not a function). It allocates memory **and** calls the constructor of the object.

```
int* p = new int(42);    // Allocate and initialize in one step
delete p;                // Release memory with delete (calls destructor)
p = nullptr;
```

Side-by-Side Comparison

Feature	`malloc`	`new`
Origin	C standard library function	C++ operator
Header needed	`#include <cstdlib>`	None (built into C++)
Return type	`void*` — must cast manually	Correct type automatically
Calls constructor	No	Yes
Calls destructor (on free)	No (`free()` used)	Yes (`delete` used)

Feature	`malloc`	`new`
Initialization	Not done automatically	Done via constructor
On failure	Returns `nullptr`	Throws `std::bad_alloc` exception
Pairing operator	`free()`	`delete` / `delete[]`
Array allocation	`malloc(n * sizeof(int))`	`new int[n]`
Array deallocation	`free(arr)`	`delete[] arr`
Type safety	Requires explicit cast	Fully type-safe
Preferred in C++	Avoid	Always use

Code Comparison

```
#include <cstdlib>
#include <iostream>
using namespace std;

class Student {
public:
    string name;
    Student() { cout << "Constructor called\n"; }
    ~Student() { cout << "Destructor called\n"; }
};

int main() {

    // --- malloc: does NOT call constructor ---
    Student* s1 = (Student*)malloc(sizeof(Student));
    // Constructor NOT called — s1->name is uninitialized garbage!
    free(s1);
    // Destructor NOT called — resources not cleaned up!

    cout << "---\n";

    // --- new: calls constructor and destructor ---
    Student* s2 = new Student();
    // Constructor IS called automatically
    delete s2;
    // Destructor IS called automatically
}
```

Output:

```
---
Constructor called
Destructor called
```

> Notice `malloc`/`free` printed nothing — the constructor and destructor were never called.

Array Allocation

```
// malloc
int* arr1 = (int*)malloc(5 * sizeof(int));   // Raw bytes, no initialization
free(arr1);

// new
int* arr2 = new int[5]{1, 2, 3, 4, 5};      // Allocated and initialized
delete[] arr2;                                 // Must use delete[] not delete
```

> **Critical mistake:** Using `delete` instead of `delete[]` for arrays is **undefined behavior** — always match `new[]` with `delete[]`.

Mixing `new`/`free` or `malloc`/`delete` — Never Do This

```
int* p1 = new int(10);
free(p1);       //  WRONG — undefined behavior

int* p2 = (int*)malloc(sizeof(int));
delete p2;      //  WRONG — undefined behavior
```

Always pair:

- `new` → `delete`
- `new[]` → `delete[]`
- `malloc` → `free`
